End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning — Assignment 2 · Roll no. 2025AG05447 · November 2026
GitHub repository: https://github.com/Huzna-git/wms-dmml · Dataset: MovieLens 100K (GroupLens)

1. Problem and scope

The task was to build a reproducible data management pipeline for a recommender, not a single notebook: raw user, item and interaction data must be ingested, validated, prepared, transformed into derived datasets and features, used to train and evaluate at least two recommenders, and the whole workflow must be orchestrated. I chose the public **MovieLens 100K** dataset because it is small enough to run end-to-end in under a minute yet carries the three required entities with real quirks: 943 user profiles (age, gender, occupation, zip), 1,682 movies (title, release date, 19 genre flags) and 100,000 timestamped 1–5 ratings from September 1997 to April 1998. A seeded synthetic generator with deliberately injected noise is a second, switchable source.

2. Pipeline design

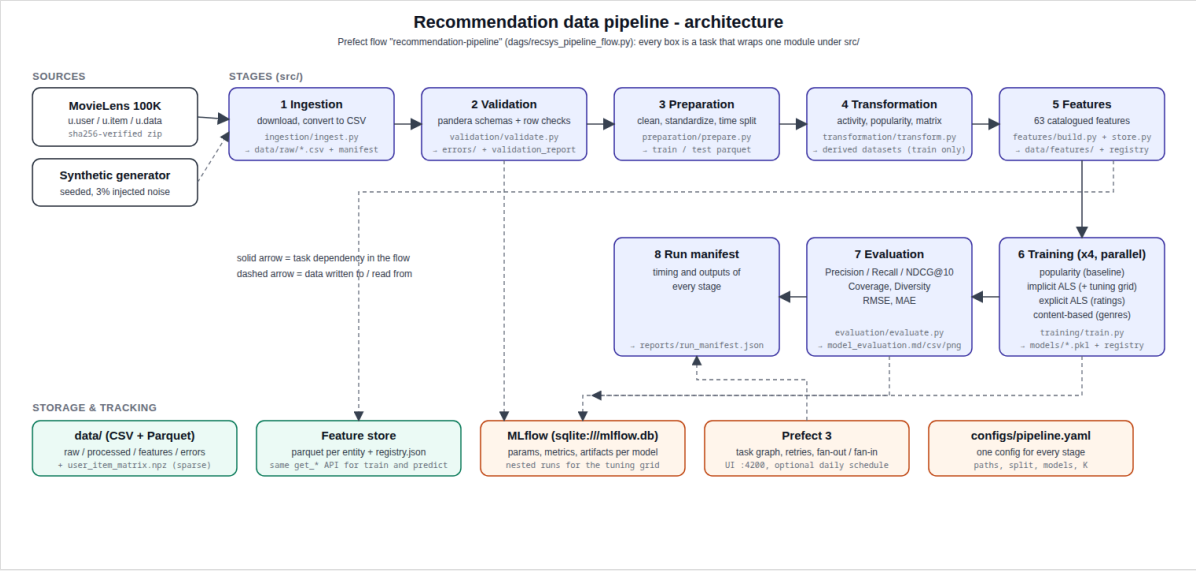


Figure 1 — Architecture. Each box is a Prefect task wrapping one module under `src/`; solid arrows are task dependencies, dashed arrows data written to or read from storage and tracking.

One YAML file (`configs/pipeline.yaml`) drives every stage; a second file is all it takes to run the same code on the synthetic data. Every stage is a pure function `run_<stage>(cfg) -> StageResult` that records timing, metrics and the files it produced, so the flow can write a complete run manifest.

#	Stage (module)	What it does	Output
1	Ingestion (`src/ingestion`)	Downloads the archive (SHA-256 verified, skipped when present) and converts the three pipe/tab-separated files into canonical CSVs. Nothing is cleaned here.	`data/raw/{users,items,interactions}.csv`, `ingestion_manifest.json`
2	Validation (`src/validation`)	Reads raw columns as strings, coerces types, runs **pandera** schemas (nulls, ranges, categories, unique keys) and row-level checks (exact and conflicting duplicates, referential integrity, date parsing, duplicate titles). Error rows are removed and written with their reasons; warnings are kept.	`data/errors/*`, `reports/validation_report.{md,json}`, `*_validated.parquet`
3	Preparation (`src/preparation`)	Standardizes formats (gender, occupation, dates, UTC timestamps), handles missing values and residual duplicates, and makes the **time-based split**: each user's history ordered by time, the last 20 % held out (users with < 5 ratings stay in train).	`users/items/interactions/train/test.parquet`, `split_manifest.json`
4	Transformation (`src/transformation`)	Derived datasets from the **train split only**: user activity summary, item popularity (`log1p(n) × mean rating`), item × genre one-hot table, sparse user–item matrix (943 × 1,682, 95 % sparse).	parquet tables, `user_item_matrix.npz` + index
5	Features (`src/features`)	30 user, 29 item and 4 interaction features declared once in a catalog, computed by pure functions and written to a file-backed **feature store** with a registry.	`data/features/*.parquet`, `registry.json`, `reports/feature_catalog.md`
6	Training (`src/training`)	Four models behind one `Recommender` interface, each an **MLflow** run with parameters, metrics and artifacts; implicit ALS is tuned on a validation split with nested runs.	`models/*.pkl`, `models/registry.json`, MLflow
7	Evaluation (`src/evaluation`)	Precision@10, Recall@10, NDCG@10, Coverage, Diversity for every model; RMSE/MAE for the rating predictors. Metrics are logged to the model's MLflow run.	`reports/model_evaluation.{md,csv,json}`, chart
8	Orchestration (`dags/`)	**Prefect 3** flow with explicit dependencies, retries on ingestion, the four training tasks fanned out in parallel, a run manifest at the end, optional daily schedule.	`reports/run_manifest.json`, Prefect UI

Design decisions. (i) *Validation before any cleaning*: the raw data is landed as-is so the checks run on the real thing, and raw strings are coerced inside the validator so a garbage value becomes a row-level `invalid_type` error instead of a crash. (ii) *Train-only derivation*: derived datasets, features and the hyper-parameter search never see the test period; recency features use the latest train timestamp as reference. (iii) *Per-user time split* rather than a random split, because a recommender is used to predict a user's *next* actions; a global cut-off is available as an option. (iv) *A declared feature catalog*: the store refuses to write a column that is not in the catalog, so documentation and data cannot drift apart, and the same `get_user_features / get_item_features` calls serve training and prediction. (v) *Two objectives, two factorization models*: top-K ranking and rating prediction are different problems, so the pipeline trains an implicit-feedback ALS for ranking and an explicit-rating ALS for RMSE/MAE instead of forcing one model to do both.

3. Tools

Python 3.12, pandas 3 / NumPy / SciPy (data and the ALS solvers, written in NumPy so that training is deterministic and dependency-free), pandera (declarative schemas), Parquet via pyarrow (columnar storage for every processed table), MLflow 3 with a SQLite backend (experiment tracking), Prefect 3 (orchestration), pytest (nine unit tests covering the validation checks, the time split, the metrics, the feature store and the model interface), matplotlib (charts) and GNU make as the entry point (`make pipeline`, `make test`, `make mlflow-ui`, `make prefect-server`, `make synthetic`).

4. Results

Validation. On MovieLens all 22 checks pass; two checks warn — one movie (id 267, "unknown") has no release date and 18 titles appear under two different ids — and the rows are kept and reported. On the synthetic dataset (39,161 interactions, 3 % corrupted) the same checks reject 1,400 rows across all seven error types: 199 non-numeric ratings, 395 missing values, 233 exact duplicates, 103 conflicting duplicate keys, 394 out-of-range ratings/timestamps, 194 unknown users and 189 unknown items; they are written to `data/synthetic/errors/interactions_errors.csv` with their reasons (screenshots 2 and 9).

Models (K = 10; a held-out rating ≥ 4 is relevant; a user's train items are excluded from their list; 908 of 943 test users have at least one relevant held-out item):

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity (baseline)	0.0773	0.0659	0.0957	0.0422	0.6458	1.0734	0.8587
Implicit ALS (CF, ranking)	**0.1102**	**0.1341**	**0.1476**	0.3532	**0.7010**	n/a	n/a
Explicit ALS (CF, ratings)	0.0760	0.0634	0.0903	0.2027	0.6878	**0.9744**	**0.7695**
Content-based (genres)	0.0178	0.0167	0.0232	**0.4810**	0.1305	1.1246	0.8933

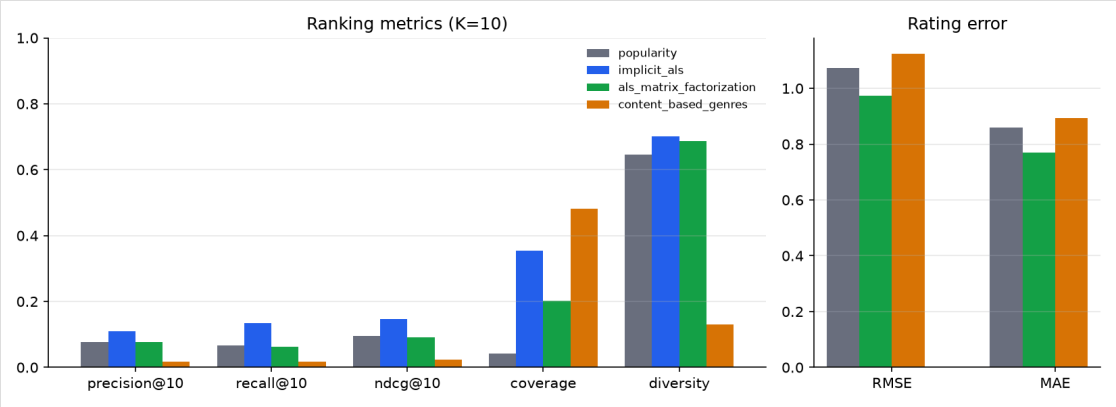


Figure 2 — Ranking metrics (left) and rating error (right) per model.

The personalized implicit ALS beats the popularity baseline on every ranking metric: NDCG@10 +54 %, recall doubled, and it recommends 35 % of the catalogue instead of 4 %. Its hyper-parameters (20 factors, α = 5) were picked from a 2 × 3 grid on a validation split carved from train by the same time rule (every candidate is a nested MLflow run; screenshot 15). The explicit ALS is the best rating predictor (RMSE 0.974 against 1.073 for the item-average baseline) but ranks no better than popularity: ranking by predicted rating favours obscure items with a few high ratings, which is exactly why a separate implicit model was added — the popularity prior in its ranking score only mitigates the effect. The content-based model is weak on accuracy but has the widest coverage and, as expected from a genre-similarity model, the lowest intra-list diversity. The absolute numbers are modest because the split is strict (each user's *latest* 20 %, with taste drift and a 4-star relevance threshold); the comparison between models is what matters.

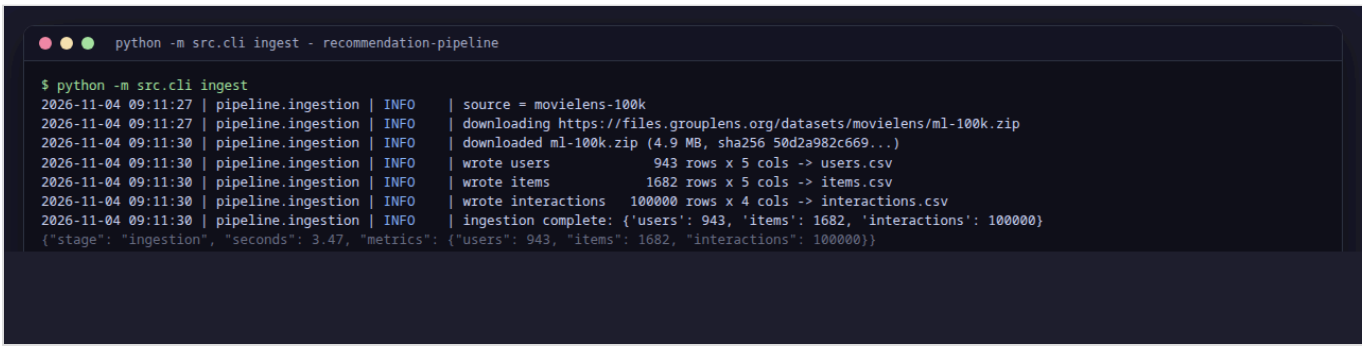
Orchestration. The Prefect run completes in about 11 s with all 11 task runs successful; the graph in screenshot 11 shows the linear data stages, the fan-out into the four training tasks and the fan-in to evaluation. Two runs give identical metrics (fixed seed, deterministic solvers).

5. Limitations and future work

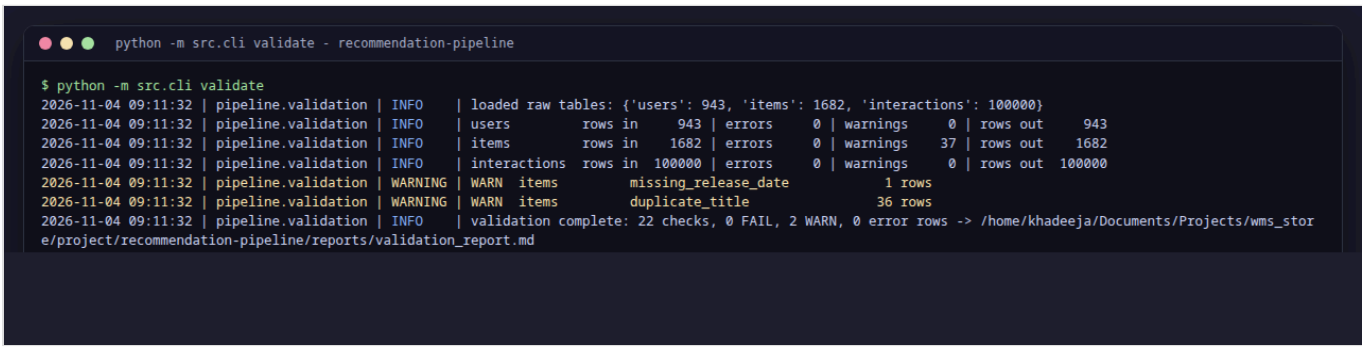
Cold-start users and items only get fallbacks (popularity for users, nothing learned for unseen items); the feature store is a set of Parquet files with a registry, not a service with point-in-time joins; evaluation is offline only, with binary relevance and no significance tests; there is no model registry or serving step after evaluation; MovieLens 100K is small and from 1998, so the figures do not transfer to a production catalogue; and the ALS implementations loop over users in Python, which is fine at this scale but would need a compiled solver (e.g. `implicit`) at millions of interactions. Natural next steps are a hybrid that blends the implicit ALS score with the genre profile, incremental feature refresh on a schedule, and a batch-scoring task that writes top-K lists for every user.

Appendix A — Execution screenshots

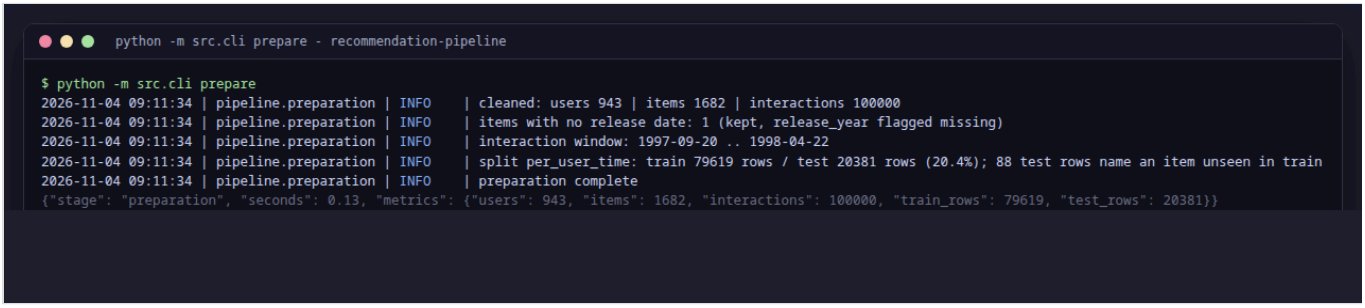
Every stage was run with the commands in the README; each picture is the complete terminal output of that run, captured by `scripts/capture_screenshots.py`, followed by the Prefect and MLflow UIs for the orchestrated run.



Screenshot 1 — Stage 1, ingestion: download verified by checksum, three raw CSVs written with a manifest.



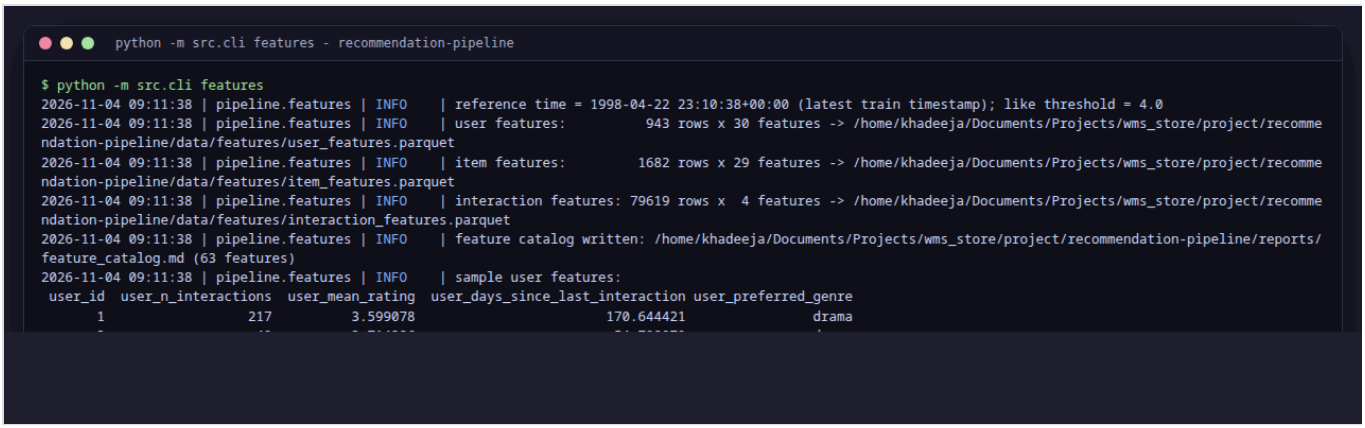
Screenshot 2 — Stage 2, validation on MovieLens: 22 checks, 0 failures, 2 warnings (missing release date, duplicate titles), report written.



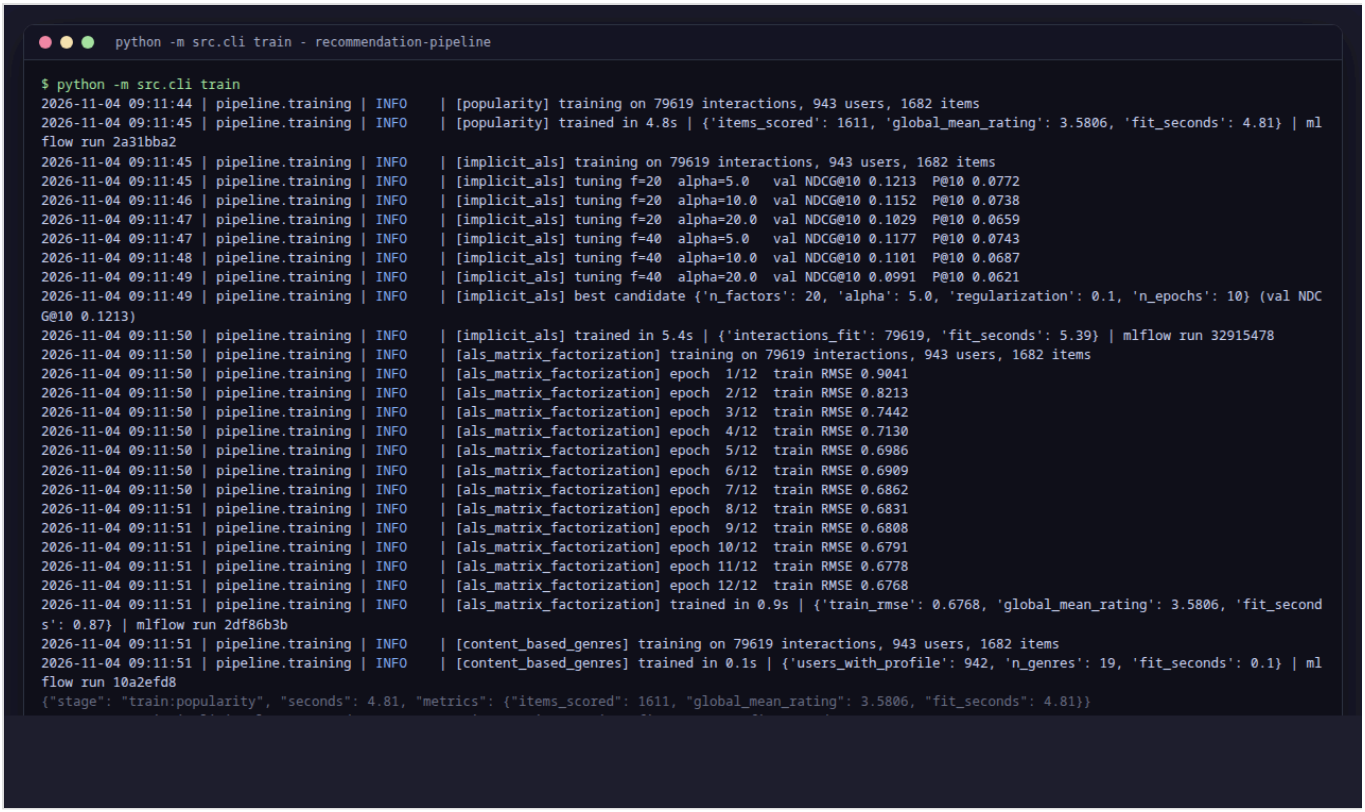
Screenshot 3 — Stage 3, preparation: cleaning and the per-user time split (79,619 train / 20,381 test rows).



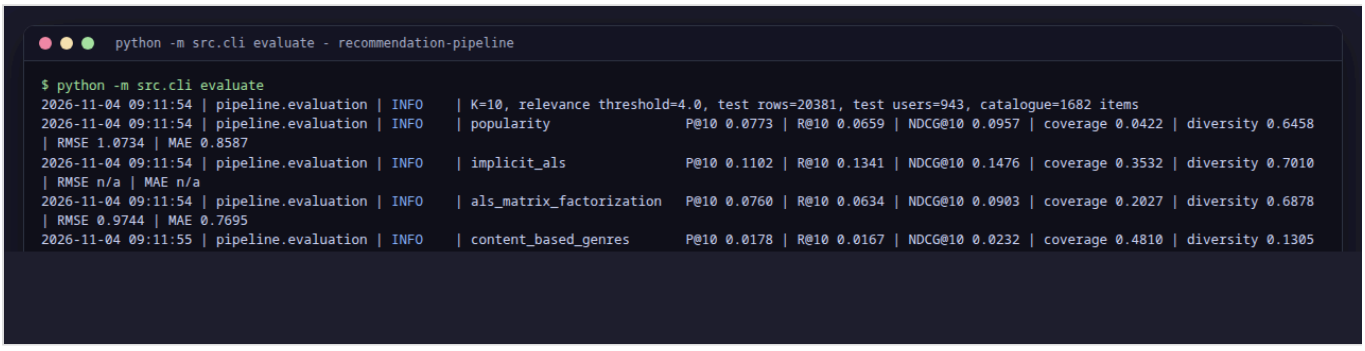
Screenshot 4 — Stage 4, transformation: user activity summary, item popularity, genre table and the sparse user–item matrix.



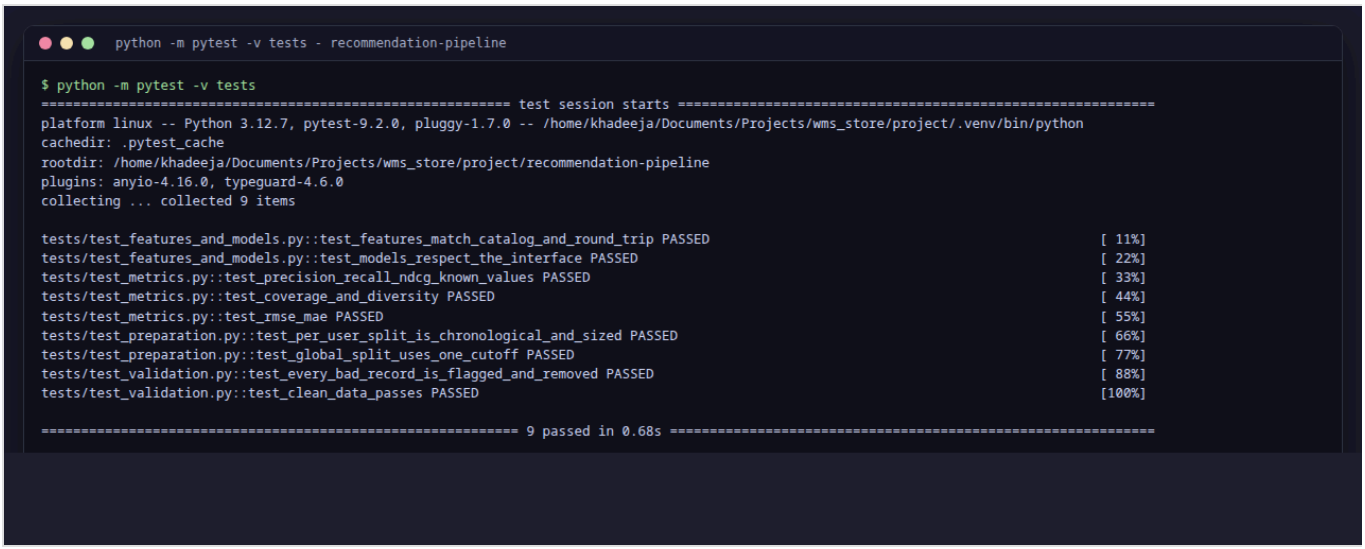
Screenshot 5 — Stage 5, feature management: 63 catalogued features written to the feature store with the reference time.



Screenshot 6 — Stage 6, training: four models, the implicit ALS tuning grid and the explicit ALS epoch RMSE, each with its MLflow run id.



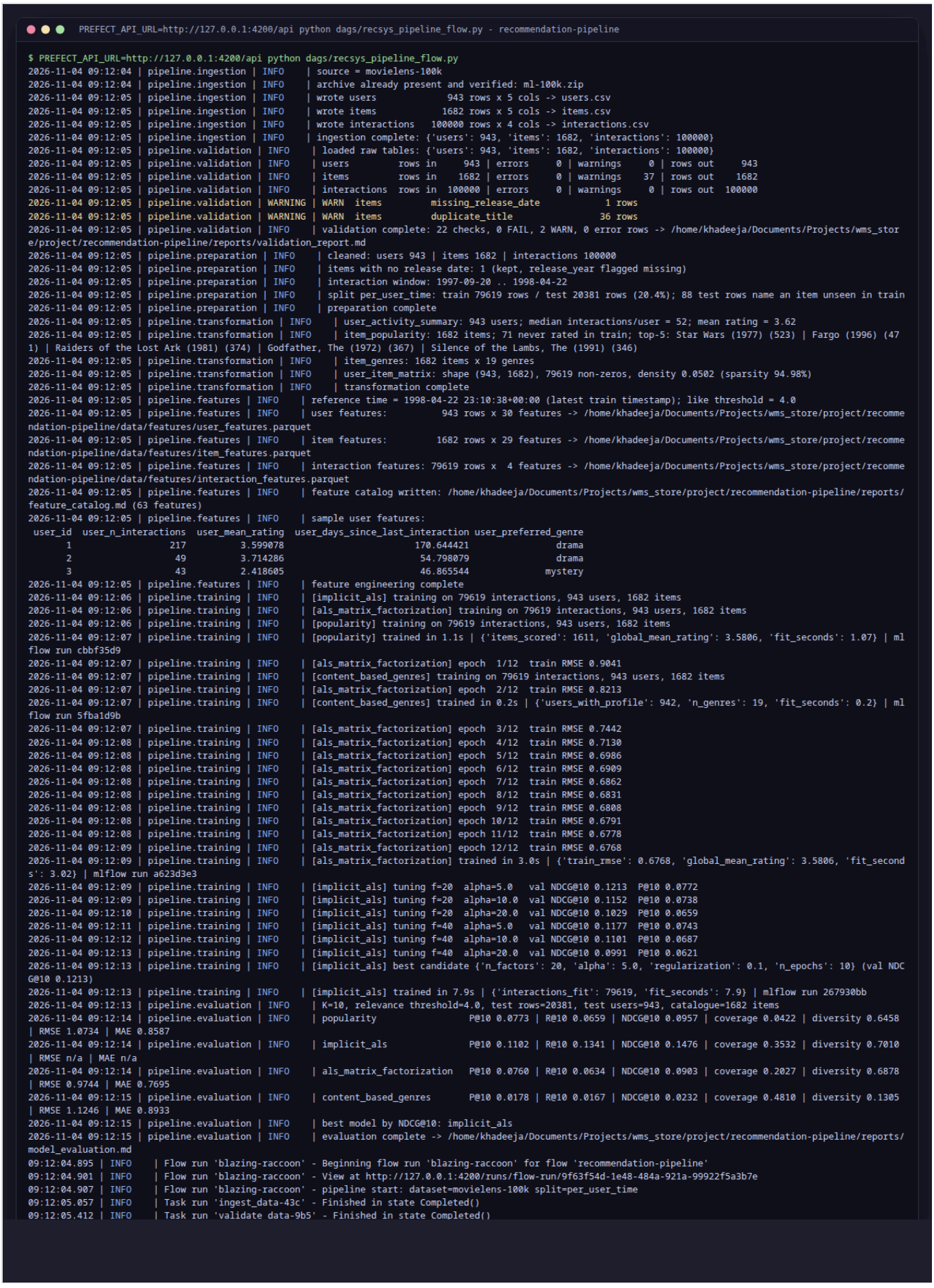
Screenshot 7 — Stage 7, evaluation: Precision/Recall/NDCG@10, coverage, diversity, RMSE/MAE per model; best model by NDCG@10.



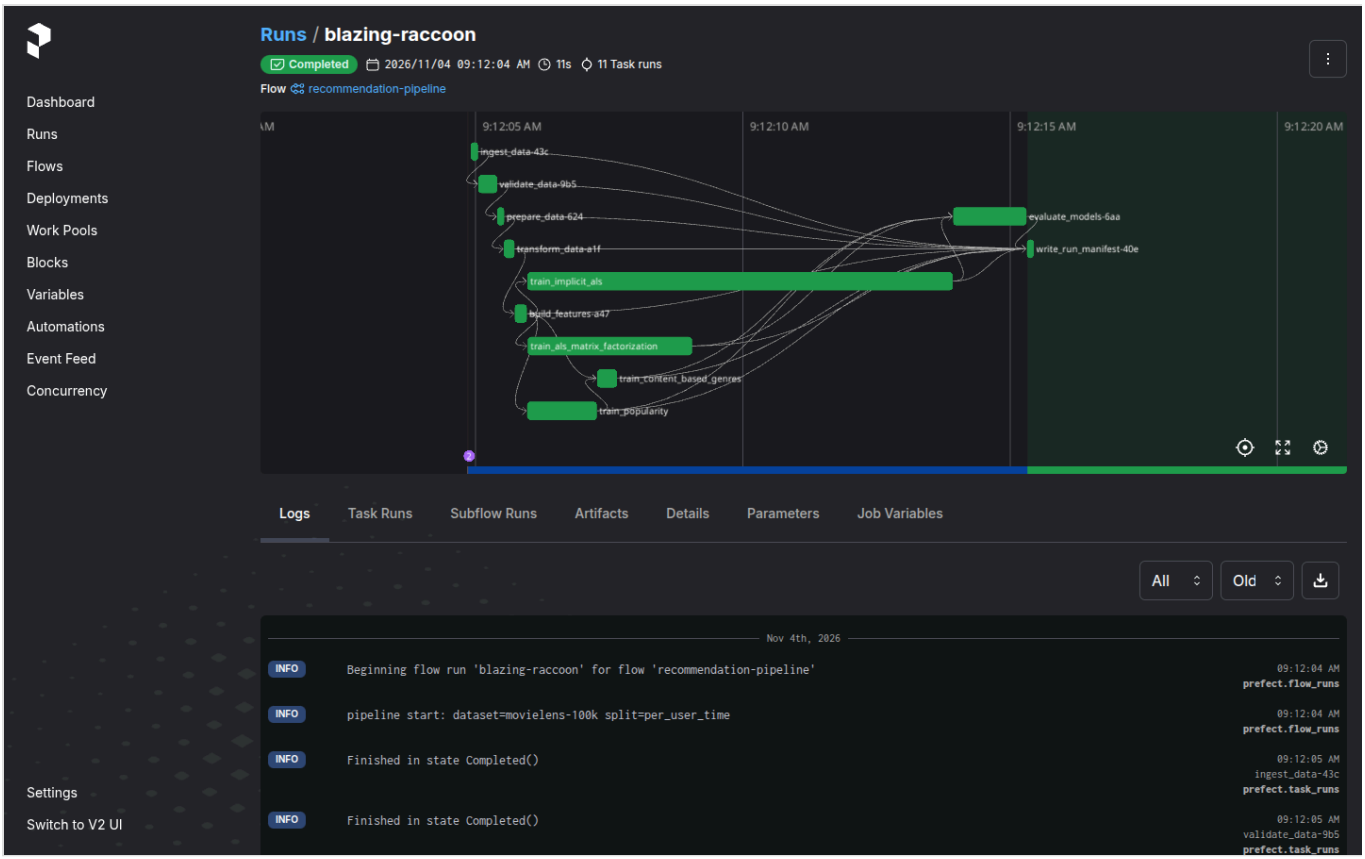
Screenshot 8 — Unit tests (validation checks, time split, metrics, feature store, model interface).



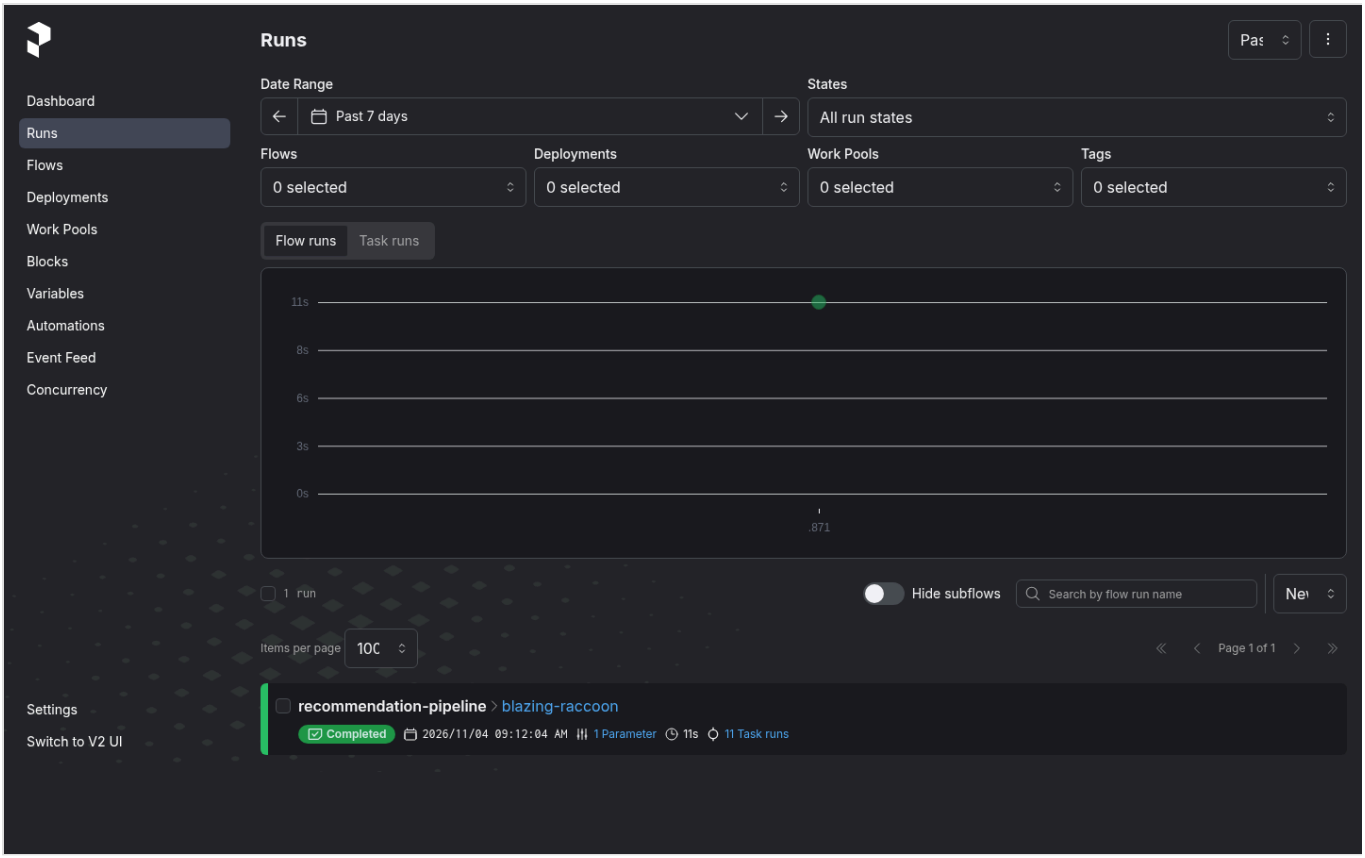
Screenshot 9 — The whole pipeline on the synthetic dataset: validation rejects 1,400 injected bad rows of seven kinds, then the remaining stages run on the clean rows.



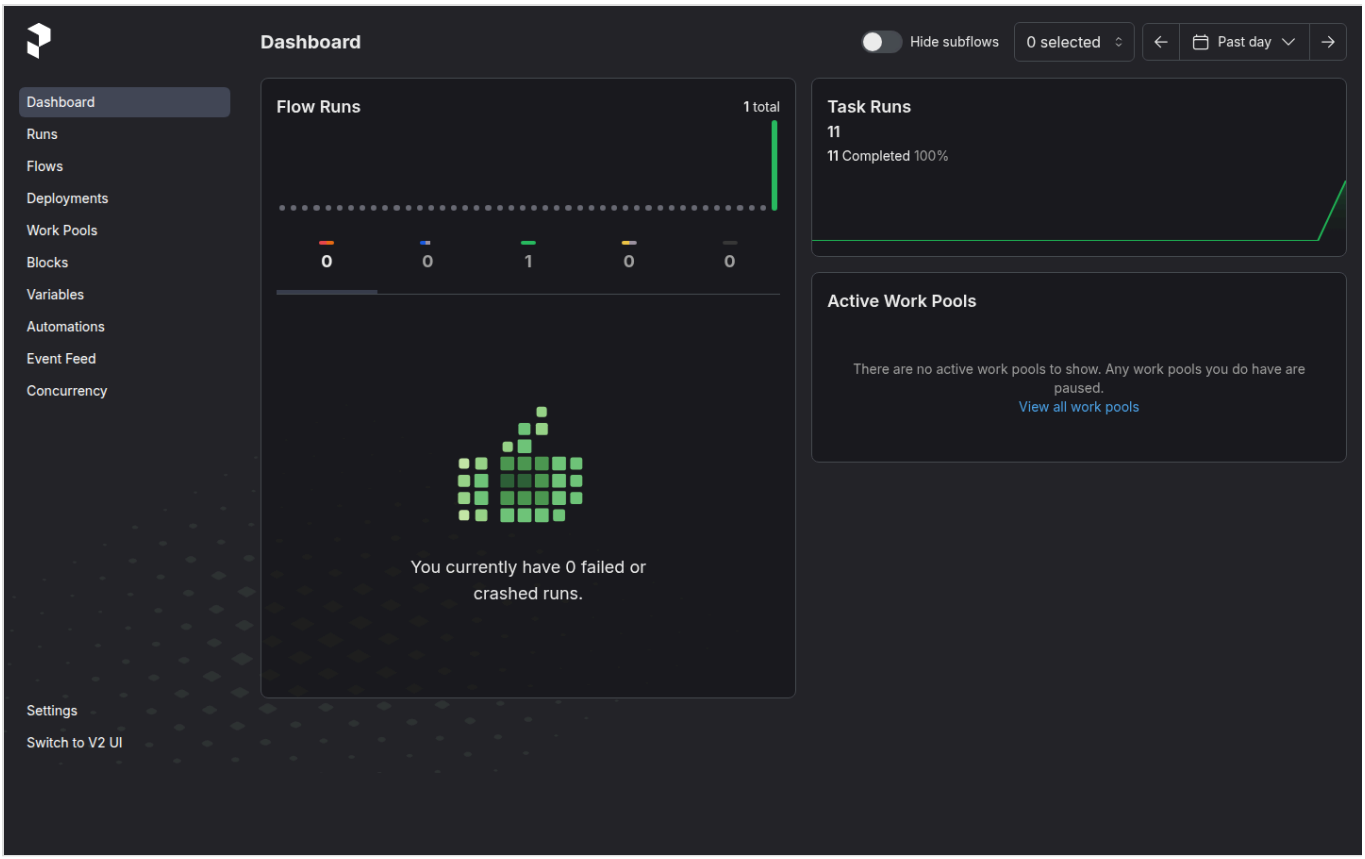
Screenshot 10 — Stage 8, orchestration: the Prefect flow run from the terminal, every task run finishing in state Completed.



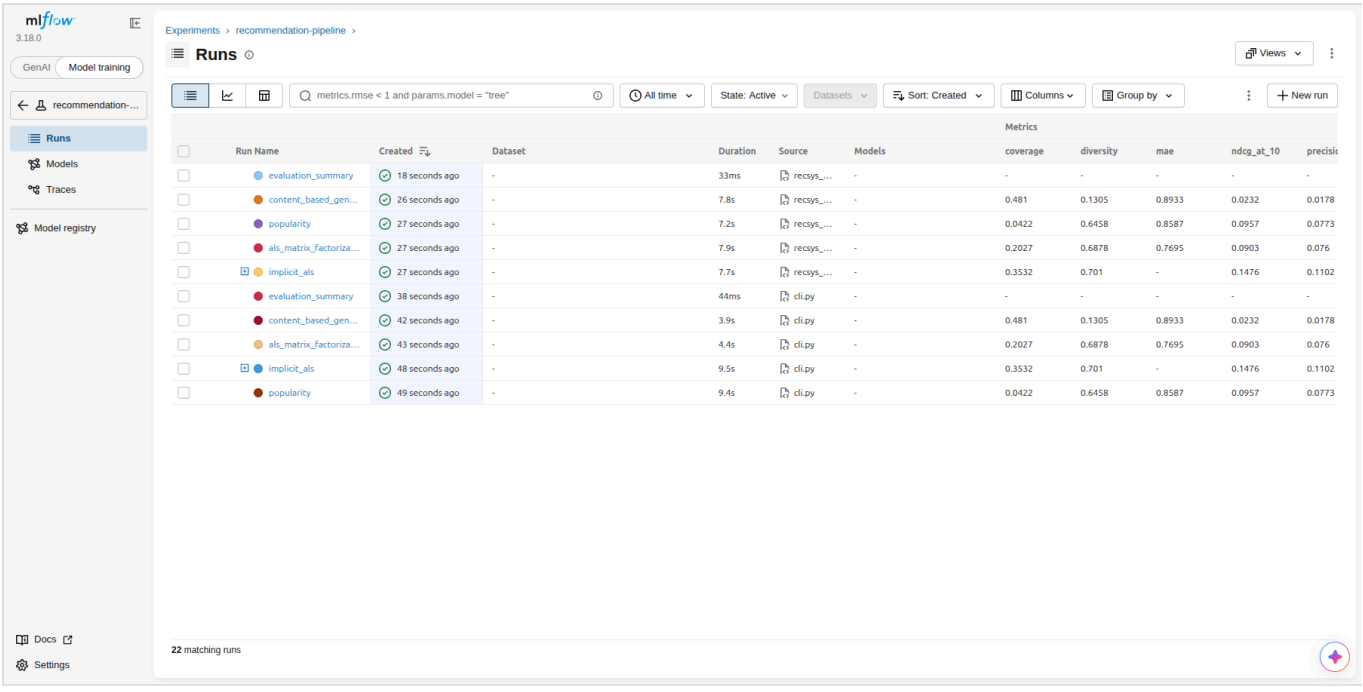
Screenshot 11 — Prefect UI, the flow run: Completed, 11 task runs; the task graph shows the chain of data stages, the parallel training tasks and the fan-in to evaluation.



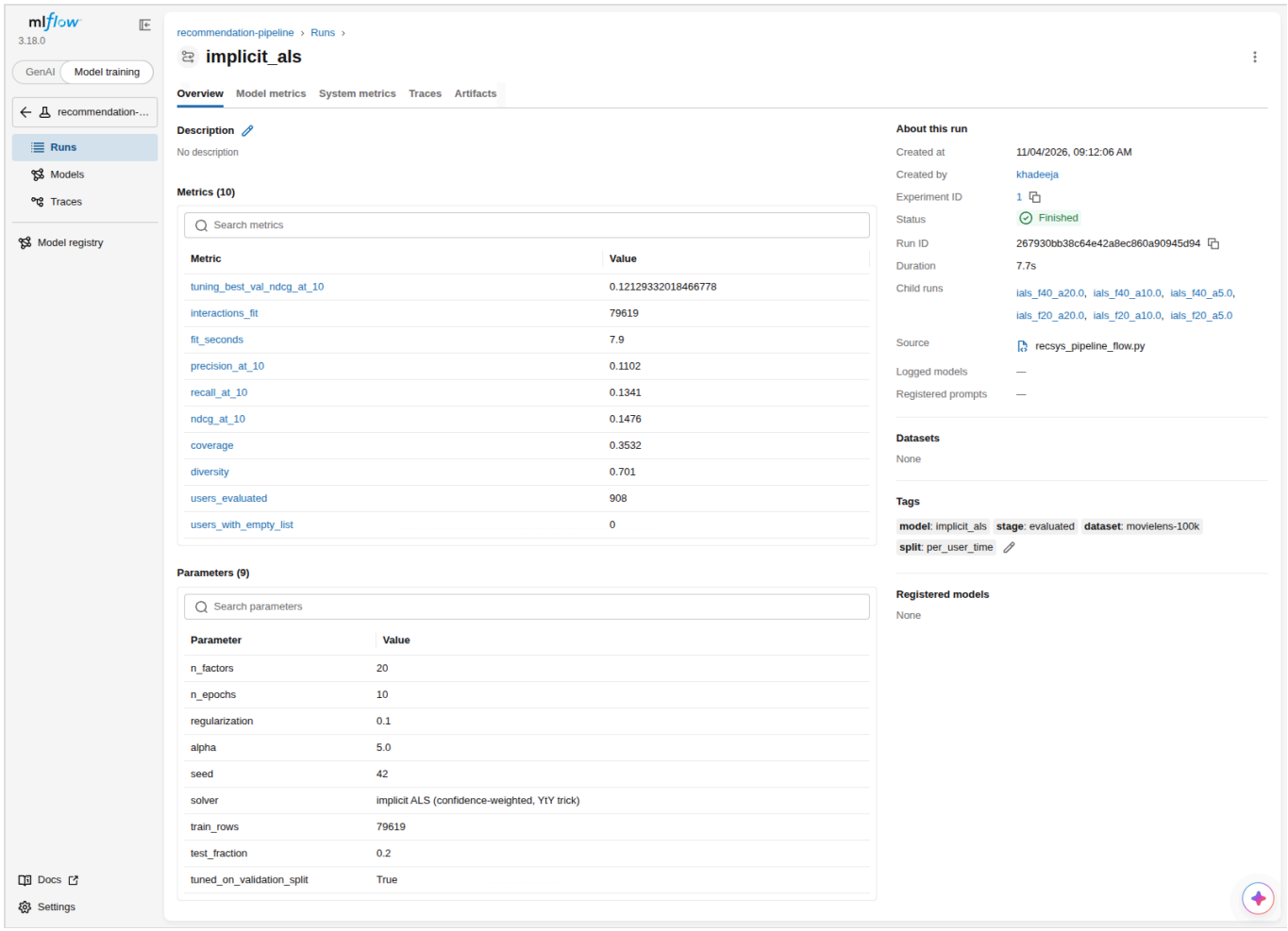
Screenshot 12 — Prefect UI, runs list.



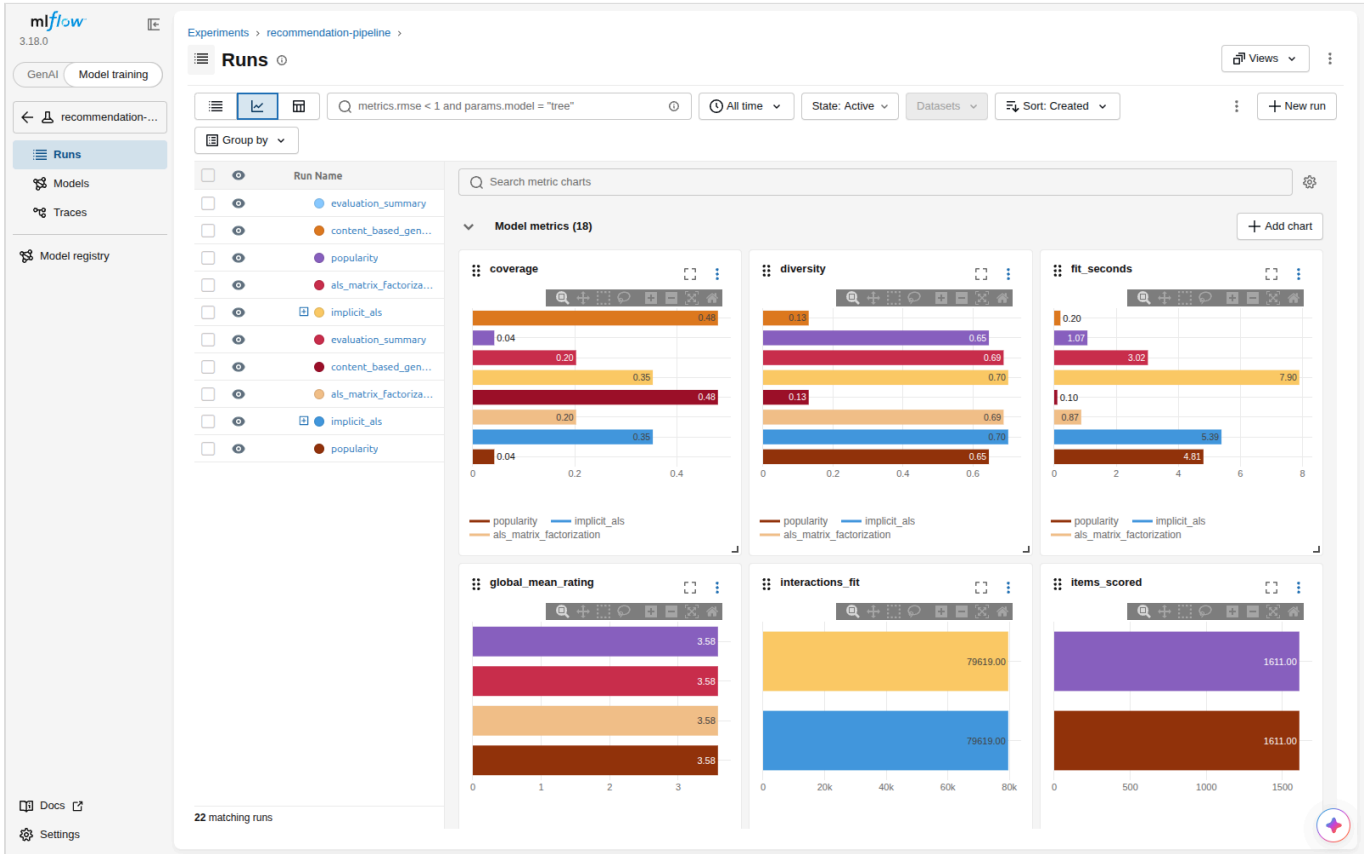
Screenshot 13 — Prefect UI, dashboard: 11 of 11 task runs completed, 0 failed.



Screenshot 14 — MLflow experiment: one run per model (plus the evaluation summary) for the stage-by-stage run (source `cli.py`) and for the orchestrated run (source `recsys_pipeline_flow.py`), with the evaluation metrics as columns.



Screenshot 15 — MLflow run page of the implicit ALS model: evaluation metrics, parameters (including the tuned values) and the six nested tuning runs.



Screenshot 16 — MLflow chart view comparing the runs' metrics.

Appendix B — Running it

```
 git clone https://github.com/Huzna-git/wms-dmml && cd wms-dmml
python -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
make pipeline        # Prefect flow, end to end            make test      # unit tests
make stages          # every stage in order, no Prefect    make synthetic # the synthetic
dataset
make mlflow-ui       # http://127.0.0.1:5000               make prefect-server  #
http://127.0.0.1:4200
```